

SOC Network Traffic Analysis Lab

Wireshark Packet Analysis Portfolio Project

Project: Project 2 - SOC/SIEM Lab: Wireshark Network Analysis

Environment: Windows workstation, Wireshark 4.6.8, Npcap

Focus: DNS, TCP, TLS, ARP, ICMP, packet filtering, TCP streams, and evidence-based SOC investigation

This project documents hands-on network traffic analysis performed on a Windows workstation. The objective was to capture normal network activity, isolate relevant protocols, follow complete network conversations, and practice making security conclusions based on packet evidence rather than assumptions.

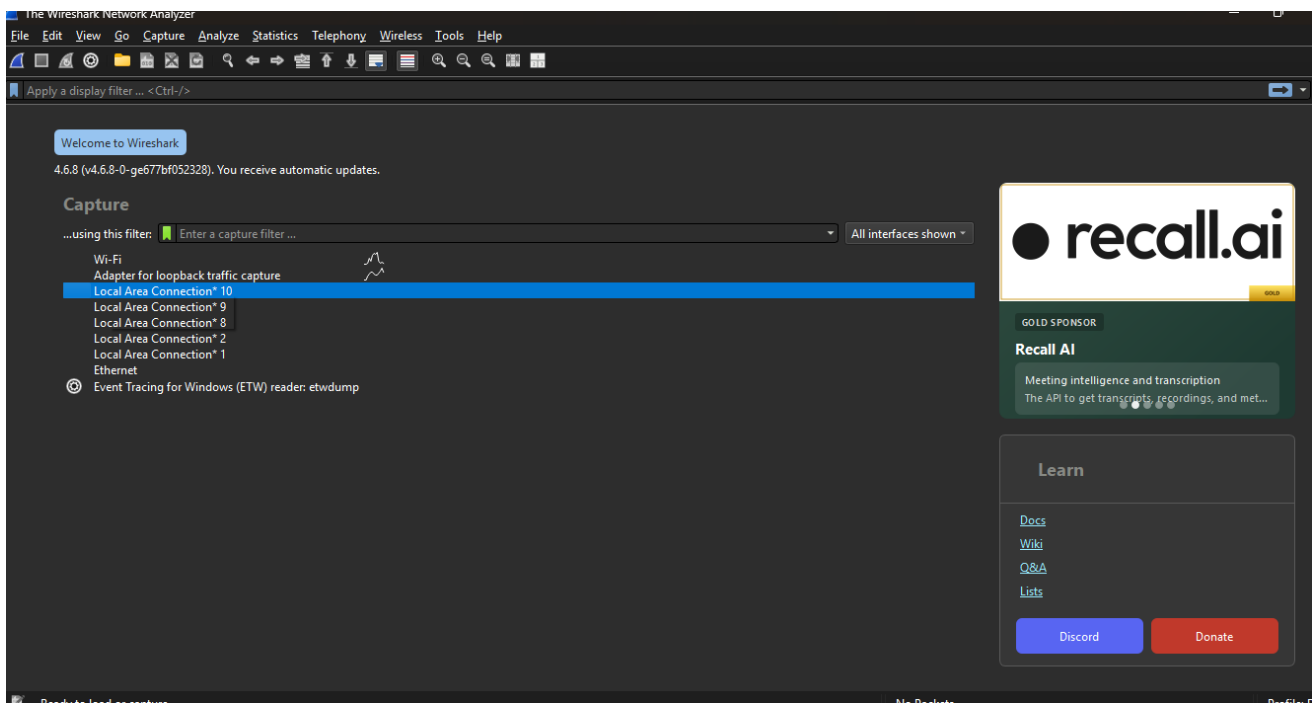


Figure 1. Wireshark capture interface used for the lab.

1. Objectives and Skills Demonstrated

- Captured live network traffic from the active Wi-Fi interface using Wireshark and Npcap.
- Applied display filters including dns, tcp, arp, icmp, ip.addr, ip.src, ip.dst, and tcp.stream.
- Analyzed DNS A and AAAA queries, responses, CNAME chains, and IPv4 resolution.
- Identified TCP three-way handshakes and distinguished graceful FIN-based closure from RST resets.
- Examined TLS Client Hello and Server Hello messages, SNI, TLS 1.3, and a negotiated cipher suite.
- Observed ARP requests/replies mapping IPv4 addresses to MAC addresses on the local network.
- Generated ICMP traffic with ping and analyzed Echo Request/Echo Reply behavior and TTL.
- Completed a final SOC-style investigation of an external TCP/443 connection and documented findings conservatively.

2. DNS Investigation

The capture was filtered with **dns** to isolate Domain Name System traffic. An A query for **ecs.office.com** was examined. The request used UDP destination port 53, and Wireshark correlated the request with its response.

No.	Time	Source	Destination	Protocol	Length	Info
123	7.496772400	fe80::453f:11ef:924...	fe80::1a60:41ff:fe9...	DNS	94	Standard query 0xc0621 A ecs.office.com
124	7.497146200	fe80::453f:11ef:924...	fe80::1a60:41ff:fe9...	DNS	94	Standard query 0xc4a0 AAAA ecs.office.com
125	7.525817800	fe80::1a60:41ff:fe9...	fe80::453f:11ef:924...	DNS	275	Standard query response 0xc0621 A ecs.office.com CNAME ecs.office.trafficmanager.net CNAME dual-s-0005-offi...
126	7.527837600	fe80::1a60:41ff:fe9...	fe80::453f:11ef:924...	DNS	299	Standard query response 0xc4a0 AAAA ecs.office.com CNAME ecs.office.trafficmanager.net CNAME dual-s-0005-o...
160	8.003885100	fe80::453f:11ef:924...	fe80::1a60:41ff:fe9...	DNS	112	Standard query 0xd1cc A mobile.events.data.microsoft.com
161	8.003581500	fe80::453f:11ef:924...	fe80::1a60:41ff:fe9...	DNS	112	Standard query 0xa9dd AAAA mobile.events.data.microsoft.com
165	8.009619600	192.168.12.219	192.168.12.1	DNS	92	Standard query 0xa9dd AAAA mobile.events.data.microsoft.com
166	8.070993900	192.168.12.219	192.168.12.1	DNS	92	Standard query 0xd1cc A mobile.events.data.microsoft.com
167	8.073831800	fe80::1a60:41ff:fe9...	fe80::453f:11ef:924...	DNS	290	Standard query response 0xa9dd AAAA mobile.events.data.microsoft.com CNAME mobile.events.data.trafficmanag...
168	8.073833400	fe80::1a60:41ff:fe9...	fe80::453f:11ef:924...	DNS	239	Standard query response 0xd1cc A mobile.events.data.microsoft.com CNAME mobile.events.data.trafficmanag...
170	8.075033500	192.168.12.1	192.168.12.219	DNS	222	Standard query response 0xd1cc A mobile.events.data.microsoft.com CNAME mobile.events.data.trafficmanag...
180	8.116362900	fe80::453f:11ef:924...	fe80::1a60:41ff:fe9...	DNS	106	Standard query 0x502c A officeclient.microsoft.com
181	8.116887200	fe80::453f:11ef:924...	fe80::1a60:41ff:fe9...	DNS	106	Standard query 0x107f AAAA officeclient.microsoft.com
186	8.123897200	192.168.12.1	192.168.12.219	DNS	267	Standard query response 0xa9dd AAAA mobile.events.data.microsoft.com CNAME mobile.events.data.trafficmanag...
190	8.148833600	fe80::453f:11ef:924...	fe80::1a60:41ff:fe9...	DNS	106	Standard query 0xa82f A support.content.office.net
191	8.149837500	fe80::453f:11ef:924...	fe80::1a60:41ff:fe9...	DNS	106	Standard query 0xed3e AAAA support.content.office.net
193	8.155133600	fe80::1a60:41ff:fe9...	fe80::453f:11ef:924...	DNS	377	Standard query response 0x502c A officeclient.microsoft.com CNAME config.officeapps.live.com CNAME prod.co...
194	8.155135300	fe80::1a60:41ff:fe9...	fe80::453f:11ef:924...	DNS	425	Standard query response 0x107f AAAA officeclient.microsoft.com CNAME config.officeapps.live.com CNAME prod...

Frame 123: Packet, 94 bytes on wire (752 bits), 94 bytes captured (752 bits) on interface \Dev...
Ethernet II, Src: VMC_2a:67:38 (dc:4b:a1:2a:67:38), Dst: Arcadyan_9e:33:3a (18:60:41:9e:33:3a)
Internet Protocol Version 6, Src: fe80::453f:11ef:924f:1eff, Dst: fe80::1a60:41ff:fe9e:333a
User Datagram Protocol, Src Port: 57786, Dst Port: 53
Domain Name System (query)
Transaction ID: 0xc0621
Flags: 0x0100 Standard query
Questions: 1
Answer RRs: 0
Authority RRs: 0
Additional RRs: 0
Queries
[Response In: 125]

Figure 2. DNS query details showing the linked response packet.

The response contained CNAME records followed by A records. This demonstrated that CNAMEs can form a resolution chain, while the final A records provide IPv4 addresses. A separate investigation of **officeclient.microsoft.com** followed a multi-step CNAME chain and resolved to multiple IPv4 addresses, including 52.110.1.191, 52.110.1.136, 52.110.1.201, and 52.110.1.216.

```

queries
  > officeclient.microsoft.com: type A, class IN
Answers
  > officeclient.microsoft.com: type CNAME, class IN, cname config.officeapps.live.com
  > config.officeapps.live.com: type CNAME, class IN, cname prod.configsvc1.live.com.akadns.net
  > prod.configsvc1.live.com.akadns.net: type CNAME, class IN, cname us.configsvc1.live.com.akadns.net
  > us.configsvc1.live.com.akadns.net: type CNAME, class IN, cname config-prod-weightedww.trafficmanager.net
  > config-prod-weightedww.trafficmanager.net: type CNAME, class IN, cname atm.office.mira.tm.svc.cloud.microsoft
  > atm.office.mira.tm.svc.cloud.microsoft: type A, class IN, addr 52.110.1.191
  > atm.office.mira.tm.svc.cloud.microsoft: type A, class IN, addr 52.110.1.136
  > atm.office.mira.tm.svc.cloud.microsoft: type A, class IN, addr 52.110.1.201
  > atm.office.mira.tm.svc.cloud.microsoft: type A, class IN, addr 52.110.1.216
[Request In: 180]
[Time: 38.770700 milliseconds]

```

Figure 3. CNAME chain followed by final A records.

Key DNS Takeaways

- A record: requests/maps an IPv4 address.
- AAAA record: requests/maps an IPv6 address.
- CNAME record: points one hostname to another hostname; the chain may continue before reaching address records.
- DNS commonly uses UDP port 53 but can also operate over TCP port 53.
- A DNS query can travel over IPv6 while asking for an IPv4 A record; transport addressing and DNS record type are separate concepts.

3. TCP Connection Analysis

TCP traffic was isolated using the **tcp** filter. A complete three-way handshake was identified by verifying the same IP/port pair in both directions: SYN, SYN-ACK, then ACK.

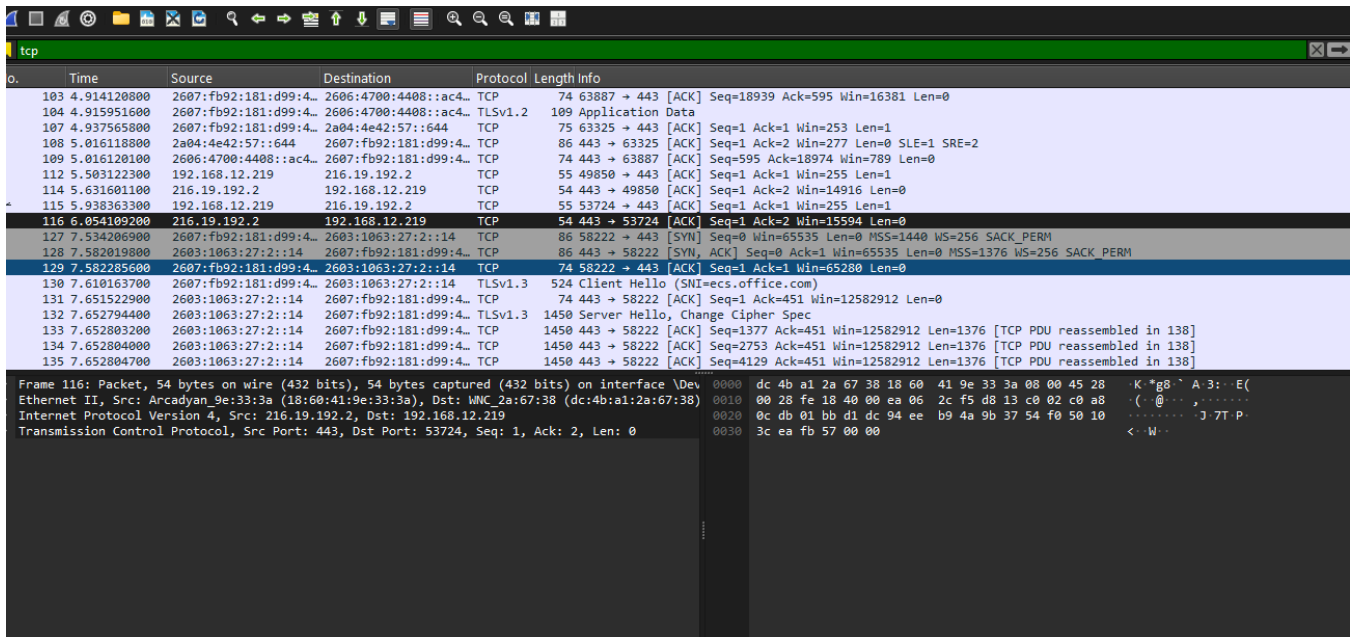


Figure 4. TCP traffic containing a three-way handshake and subsequent TLS activity.

The analysis reinforced that TCP is connection-oriented. The client uses an ephemeral source port and connects to the server's service port. For HTTPS, the expected server port is TCP 443.

TCP Stream Analysis

The **tcp.stream** feature was used to isolate a single TCP conversation. Stream 24 confirmed that packets 258, 267, and 269 formed one handshake between 192.168.12.219:58226 and 23.7.43.150:443. The stream later ended with an RST, ACK sent by the client.

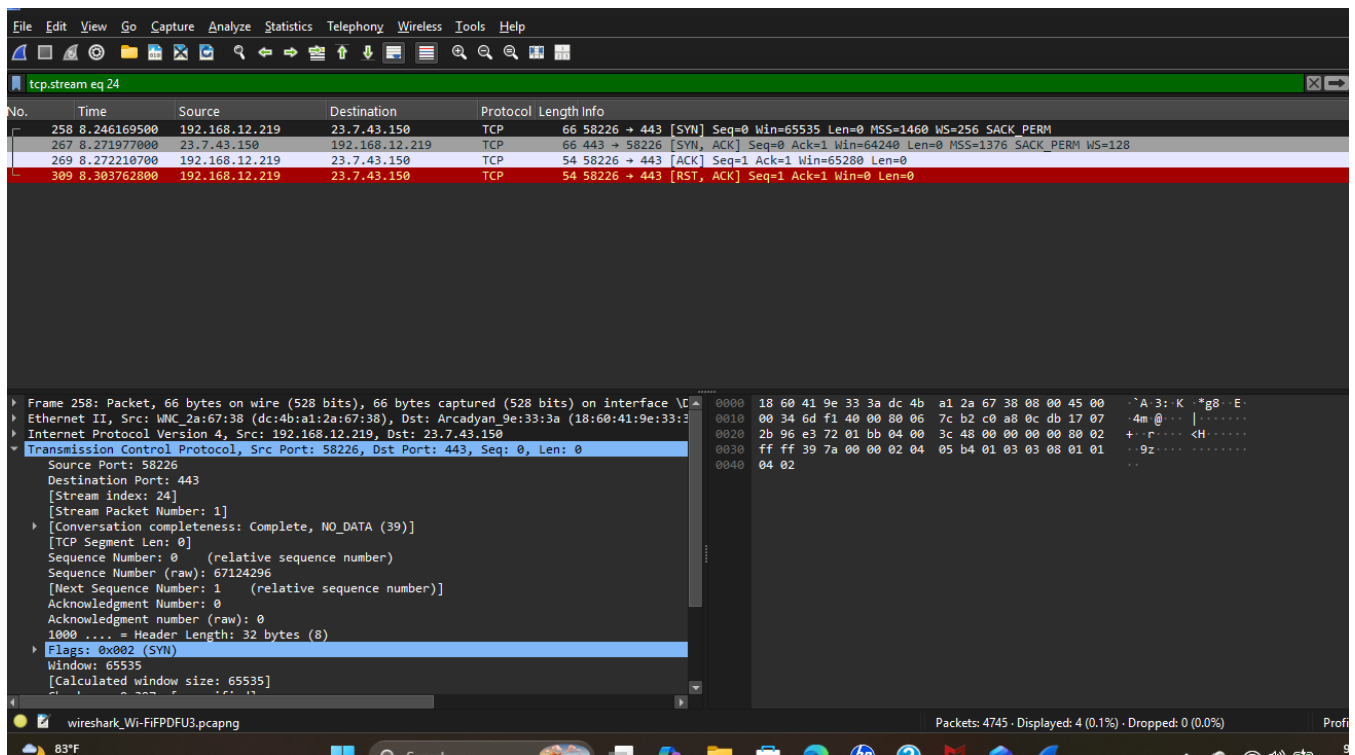


Figure 5. TCP Stream 24 isolates one conversation and shows a later RST, ACK.

A second stream demonstrated graceful connection termination. Stream 32 showed connection establishment, DNS communication over TCP port 53, and a FIN/ACK-based close. This provided a direct comparison between a graceful FIN closure and an abrupt RST reset.

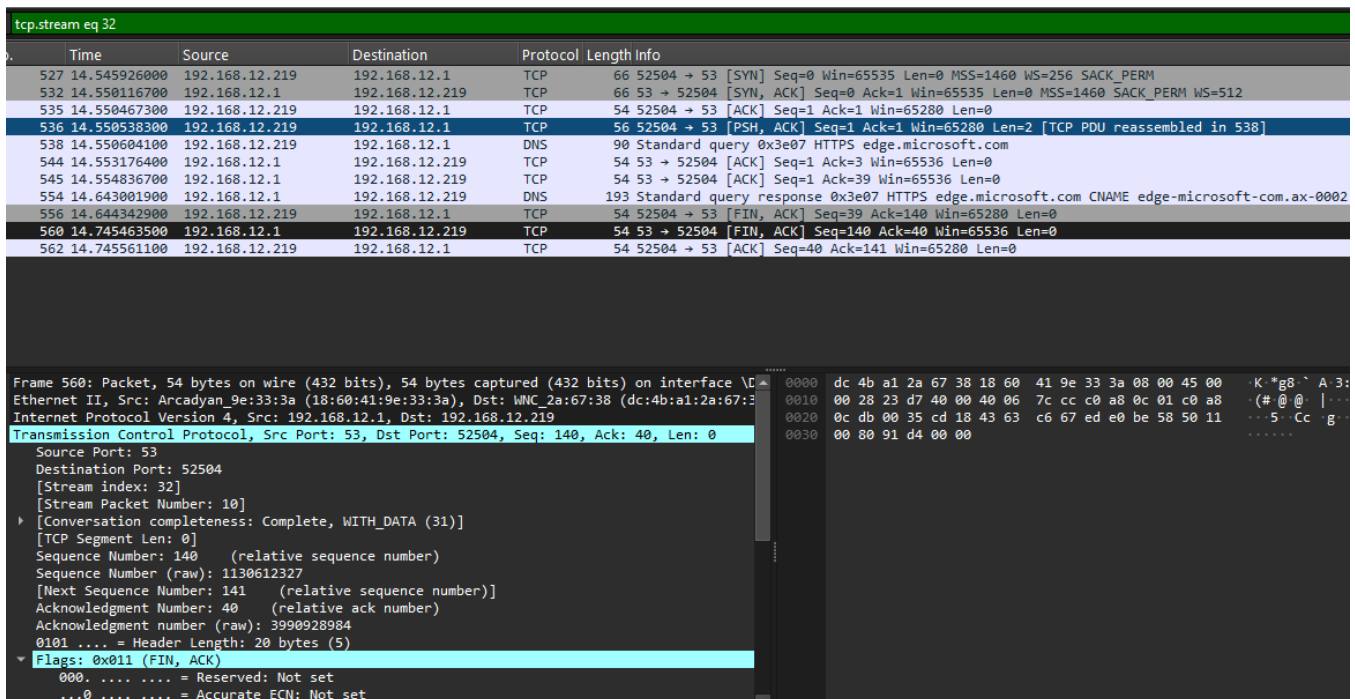


Figure 6. TCP Stream 32 showing handshake, DNS exchange, and graceful termination.

4. TLS / HTTPS Analysis

After a TCP connection to port 443 was established, TLS handshake traffic was examined. A TLS 1.3 Client Hello exposed metadata such as the Server Name Indication (SNI), supported versions, cipher suites, and key-share groups.

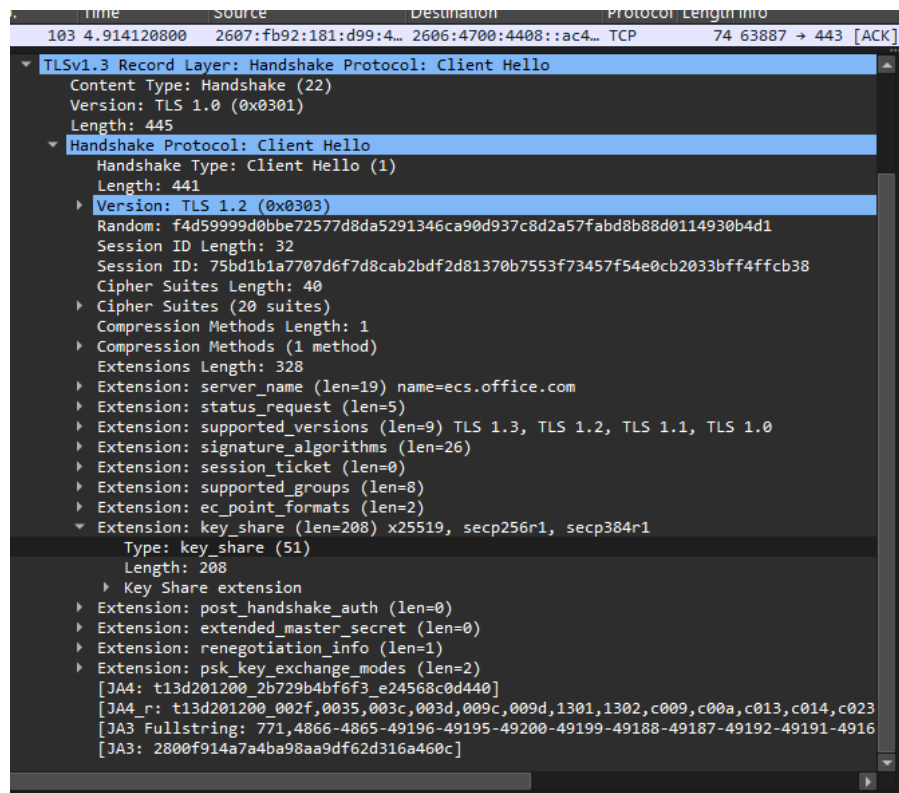


Figure 7. TLS Client Hello showing SNI, supported versions, cipher suites, and key-share options.

The Server Hello selected the cipher suite **TLS_AES_256_GCM_SHA384**. AES-256-GCM provides authenticated symmetric encryption, while SHA-384 is used within TLS 1.3 cryptographic key-derivation/handshake functions. This analysis reinforced the distinction between TCP, which establishes reliable transport, and TLS, which establishes protected communication for HTTPS.

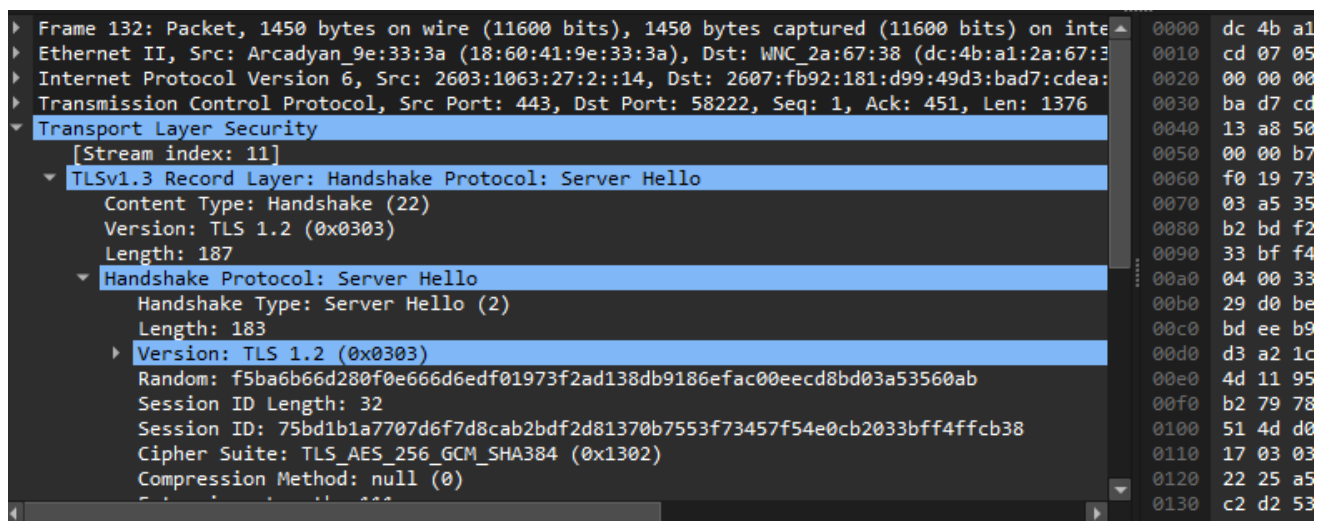


Figure 8. TLS Server Hello showing TLS_AES_256_GCM_SHA384.

5. ARP Analysis

ARP traffic was filtered with **arp**. A request from 192.168.12.219 asked, "Who has 192.168.12.1?" The reply mapped 192.168.12.1 to MAC address 18:60:41:9e:33:3a. The reverse exchange also appeared, showing the local gateway resolving the workstation's IPv4 address to its MAC address.

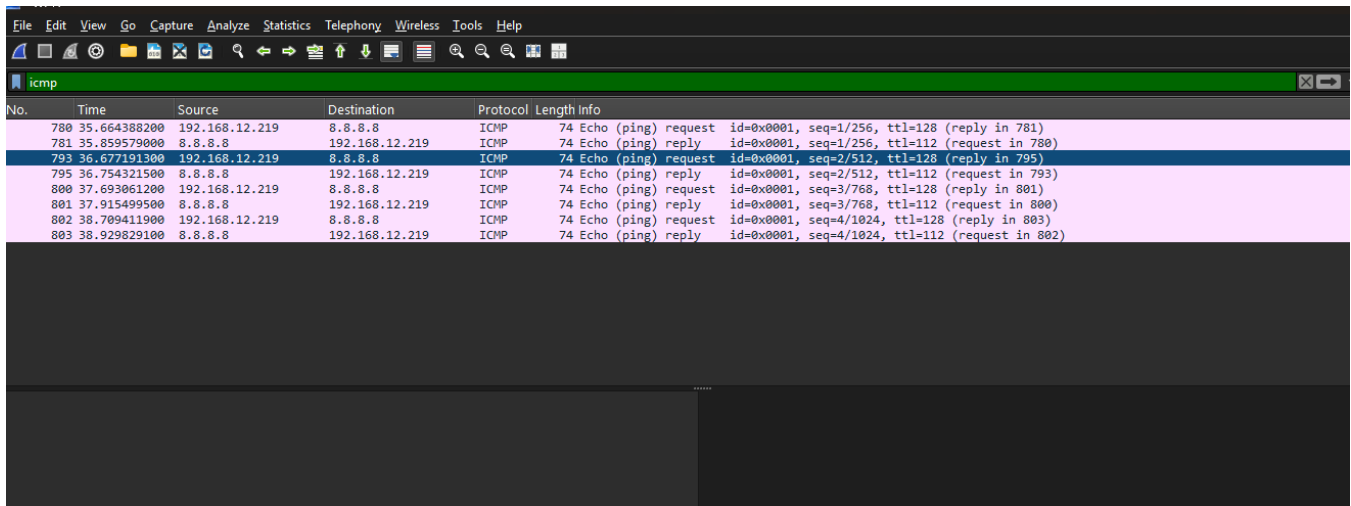
arp									
No.	Time	Source	Destination	Protocol	Length	Info			
4654	115.814462600	WNC_2a:67:38	Broadcast	ARP	42	Who has 192.168.12.148? Tell 192.168.12.219			
4668	116.948067000	WNC_2a:67:38	Broadcast	ARP	42	Who has 192.168.12.148? Tell 192.168.12.219			
4682	117.811879300	WNC_2a:67:38	Broadcast	ARP	42	Who has 192.168.12.148? Tell 192.168.12.219			
4683	118.169484500	Arcadyan_9e:33:3a	Broadcast	ARP	42	Who has 192.168.12.189? Tell 192.168.12.1			
4694	118.815407700	WNC_2a:67:38	Broadcast	ARP	42	Who has 192.168.12.148? Tell 192.168.12.219			
4703	119.602995900	Arcadyan_9e:33:3a	Broadcast	ARP	42	Who has 192.168.12.171? Tell 192.168.12.1			
4712	120.729222600	Arcadyan_9e:33:3a	Broadcast	ARP	42	Who has 192.168.12.246? Tell 192.168.12.1			
4713	120.729223400	Arcadyan_9e:33:3a	Broadcast	ARP	42	Who has 192.168.12.177? Tell 192.168.12.1			
4714	120.960776100	WNC_2a:67:38	Broadcast	ARP	42	Who has 192.168.12.148? Tell 192.168.12.219			
4715	121.815622600	WNC_2a:67:38	Broadcast	ARP	42	Who has 192.168.12.148? Tell 192.168.12.219			
4724	122.809384900	WNC_2a:67:38	Broadcast	ARP	42	Who has 192.168.12.148? Tell 192.168.12.219			
4727	123.302762300	WNC_2a:67:38	Arcadyan_9e:33:3a	ARP	42	Who has 192.168.12.1? Tell 192.168.12.219			
4730	123.903720000	Arcadyan_9e:33:3a	Broadcast	ARP	42	Who has 192.168.12.161? Tell 192.168.12.1			
4733	123.984143200	Arcadyan_9e:33:3a	WNC_2a:67:38	ARP	42	192.168.12.1 is at 18:60:41:9e:33:3a			
4739	125.849664500	Arcadyan_9e:33:3a	Broadcast	ARP	42	Who has 192.168.12.219? Tell 192.168.12.1			
4740	125.849745000	WNC_2a:67:38	Arcadyan_9e:33:3a	ARP	42	192.168.12.219 is at dc:4b:a1:2a:67:38			

Frame 49: Packet, 42 bytes on wire (336 bits), 42 bytes captured (336 bits) on interface \Device				0000	ff ff ff ff ff ff dc 4b	a1 2a 67 38 08 06 00 01		K	.*g8...
Ethernet II, Src: WNC_2a:67:38 (dc:4b:a1:2a:67:38), Dst: Broadcast (ff:ff:ff:ff:ff:ff)				0010	08 00 06 04 00 01 dc 4b	a1 2a 67 38 c0 a8 0c db		K	.*g8...
Address Resolution Protocol (request)				0020	00 00 00 00 00 00 c0 a8	0c 94		.	

Figure 9. ARP request and reply resolving IPv4 addresses to MAC addresses.

6. ICMP Analysis

A new live capture was started and **ping 8.8.8.8** was generated from the Windows command prompt. Filtering with **icmp** showed four Echo Requests and four matching Echo Replies. The workstation 192.168.12.219 sent the requests and 8.8.8.8 returned the replies.



The image shows a Wireshark packet capture window with the filter 'icmp' applied. The packet list shows four pairs of ICMP Echo (ping) request and reply packets. The first pair (780-781) has sequence number 1/256, TTL 128. The second pair (793-795) has sequence number 2/512, TTL 128. The third pair (800-801) has sequence number 3/768, TTL 128. The fourth pair (802-803) has sequence number 4/1024, TTL 128. All requests are from 192.168.12.219 to 8.8.8.8, and all replies are from 8.8.8.8 to 192.168.12.219.

No.	Time	Source	Destination	Protocol	Length	Info
780	35.664388200	192.168.12.219	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=1/256, ttl=128 (reply in 781)
781	35.859579800	8.8.8.8	192.168.12.219	ICMP	74	Echo (ping) reply id=0x0001, seq=1/256, ttl=112 (request in 780)
793	36.677191300	192.168.12.219	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=2/512, ttl=128 (reply in 795)
795	36.754321500	8.8.8.8	192.168.12.219	ICMP	74	Echo (ping) reply id=0x0001, seq=2/512, ttl=112 (request in 793)
800	37.693061200	192.168.12.219	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=3/768, ttl=128 (reply in 801)
801	37.915499500	8.8.8.8	192.168.12.219	ICMP	74	Echo (ping) reply id=0x0001, seq=3/768, ttl=112 (request in 800)
802	38.709411900	192.168.12.219	8.8.8.8	ICMP	74	Echo (ping) request id=0x0001, seq=4/1024, ttl=128 (reply in 803)
803	38.929829100	8.8.8.8	192.168.12.219	ICMP	74	Echo (ping) reply id=0x0001, seq=4/1024, ttl=112 (request in 802)

Figure 10. ICMP Echo Request and Echo Reply pairs generated with ping.

ICMP operates at the network layer and does not use TCP or UDP port numbers. TTL was interpreted as a hop-limit mechanism: routers decrement the IPv4 TTL as packets are forwarded, and packets are discarded when TTL reaches zero, helping prevent indefinite routing loops.

7. Final SOC Investigation

Scenario: investigate a workstation observed communicating with an external system over TCP port 443. The analysis used TCP filtering and then isolated **tcp.stream eq 19**.

Evidence	Finding
Client	192.168.12.219:61978
External destination	204.79.197.222:443
SYN	Packet 597
SYN-ACK	Packet 598
ACK	Packet 599
Handshake status	Successful
Expected service	HTTPS (TCP 443)
TLS	TLS 1.3
SNI	fp.msedge.net
Assessment	Not enough packet evidence to classify the connection as malicious

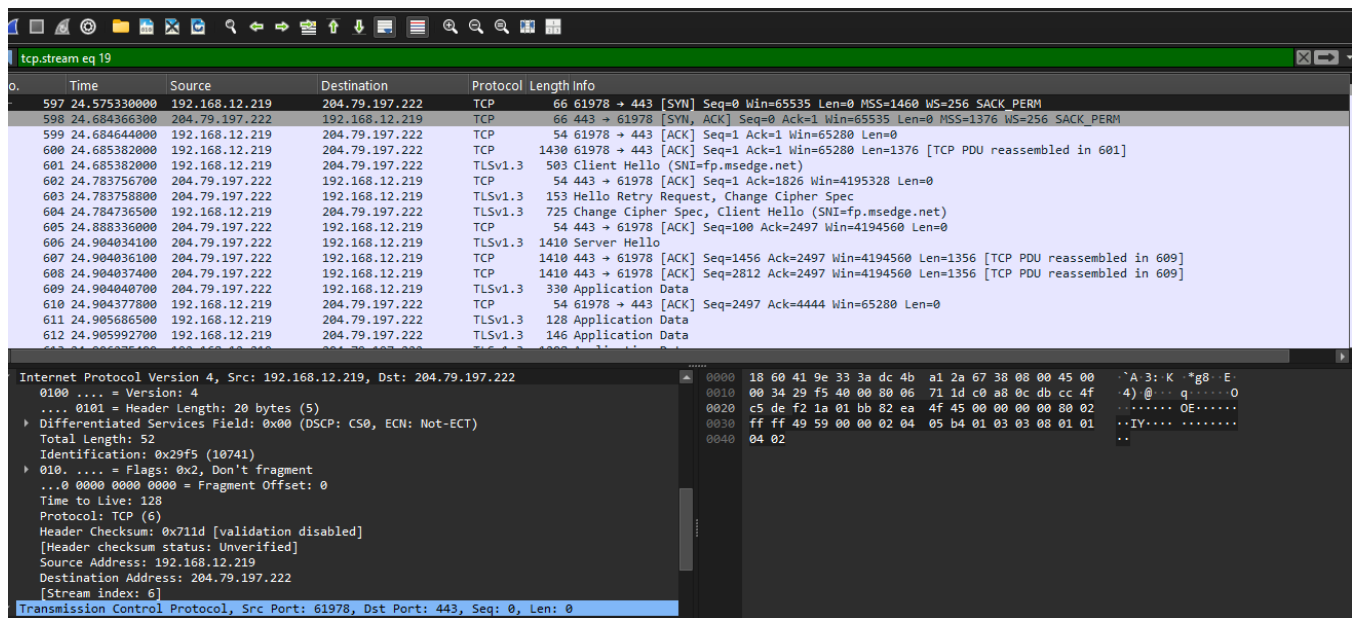


Figure 11. Final investigation: TCP Stream 19 with successful handshake and TLS 1.3 traffic.

Analyst conclusion: The packet capture proves that the workstation successfully established a TCP connection to an external host on port 443 and then exchanged TLS 1.3 traffic. The Client Hello identifies SNI fp.msedge.net. No analyzed packet provides sufficient evidence to label the connection malicious. Additional endpoint telemetry, process information, SIEM logs, or threat-intelligence context would be needed for a stronger security determination.

8. Key Lessons Learned

- Network analysis requires correlating protocol, IP address, port, flags, timing, and packet details rather than relying on a single field.
- Packets located next to each other are not automatically part of the same TCP conversation; matching endpoints/ports or tcp.stream should be used to verify.
- A successful TCP handshake proves connection establishment, not that the activity is safe.
- Encrypted HTTPS traffic still exposes useful network metadata, although application content is protected.
- Security conclusions should distinguish observed evidence from assumptions.

9. Portfolio / Resume Entry

SOC Network Traffic Analysis Lab - Wireshark

- Captured and analyzed Windows network traffic with Wireshark/Npcap, applying DNS, TCP, TLS, ARP, ICMP, IP, and TCP-stream filters.
- Investigated DNS resolution, TCP three-way handshakes, TLS 1.3 negotiation, ARP address resolution, ICMP reachability, and FIN/RST connection termination.
- Performed a SOC-style investigation of external TCP/443 traffic, validated packet evidence, identified TLS SNI, and documented an evidence-based security assessment.

Tools/Concepts: Wireshark, Npcap, TCP/IP, DNS, TLS, HTTPS, ARP, ICMP, IPv4/IPv6, OSI model, packet filtering, TCP streams, network traffic analysis, SOC investigation.

Suggested interview summary: "I built a Wireshark network-analysis lab where I captured live Windows traffic and investigated DNS, TCP, TLS, ARP, and ICMP. I followed TCP streams from handshake through termination, examined TLS negotiation and SNI, and completed a SOC-style investigation of an external HTTPS connection. The project helped me practice separating what packet evidence proves from what would require additional endpoint or SIEM context."